

Презентация по отчёту о прохождении преддипломной практики

по теме: «Разработка real-time интерфейса с поддержкой горизонтального масштабирования WebSocket-соединений для системы отслеживания задач»

Студент: Зайцев А. С., группа 4131з

Руководитель: ст. преподаватель С. А. Рогачев

Санкт-Петербург, 2026

Проблема и цель

Single-node SignalR ограничивает горизонтальное масштабирование

Проблема

Сведения о подключениях и группах SignalR хранятся в памяти процесса.

При нескольких экземплярах backend событие, созданное на одном узле, не доходит до клиента, подключенного к другому узлу.

Real-time пространство фрагментируется на независимые острова.

Цель преддипломной практики

Real-time интерфейс системы отслеживания задач, в котором WebSocket-соединения обслуживаются несколькими экземплярами app-арі, а события об изменении сущностей доставляются клиентам независимо от узла подключения.

Анализ подходов к масштабированию

Шесть рассмотренных вариантов; обоснован выбор Redis backplane

Один экземпляр backend
Нет масштабирования; один узел — единственная точка отказа

Несколько экземпляров без общего канала
События не доходят до клиентов, подключённых к другим узлам

Привязка клиента к узлу (sticky)
Клиент остаётся на одном узле, но события на другие не приходят

Внешняя шина событий
Доставка между узлами есть, но требуется отдельная схема событий

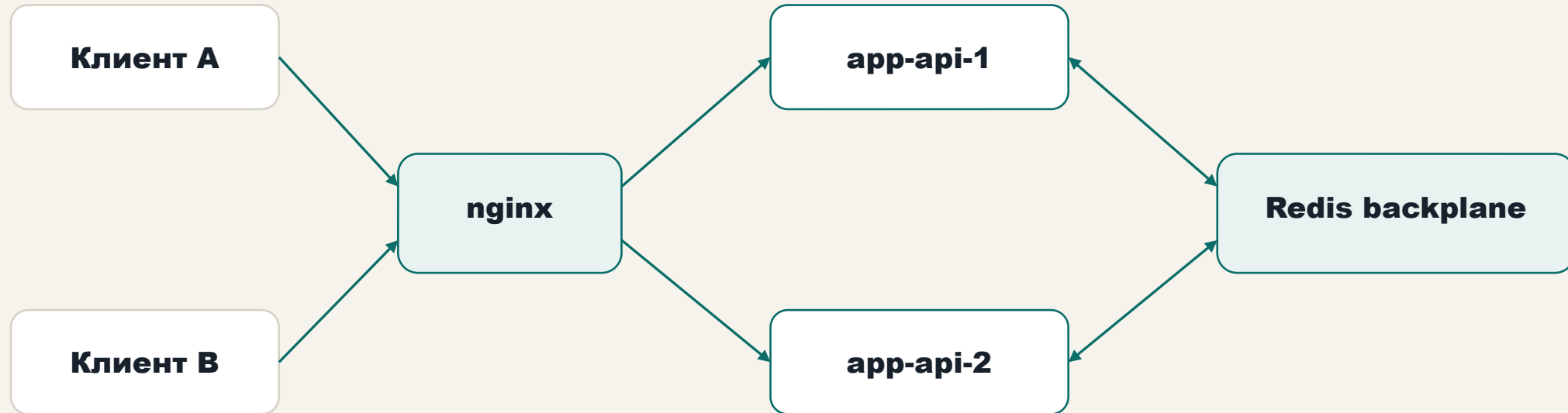
Облачный real-time сервис
Готовое решение, но снижает контроль над собственным контуром

**Redis backplane для SignalR —
выбрано**
**Штатный механизм, без
перестройки кода, подходит для
событий интерфейса**

Критерии выбора: запускается на локальном стенде, сохраняет существующий код доставки событий, штатно поддерживается в ASP.NET Core SignalR, подходит для коротких событий интерфейса и легко воспроизводится.

Целевая архитектура

Два экземпляра app-api за nginx, общий Redis backplane для межузловой доставки



Событие публикуется в Redis · доставляется обоим узлам · группы SignalR остаются локальным runtime-состоянием

Оба экземпляра подключены к общей PostgreSQL; на схеме показан только канал real-time-доставки.

Что сделано

Подключён межузловой канал, добавлена диагностика, собран стенд и клиент проверки

1 · Межузловая доставка событий

К SignalR подключён Redis backplane. Включается переменная окружения — один и тот же код работает и в режиме с межузловым каналом, и без него.

3 · Многоэкземплярный стенд

Два экземпляра backend за nginx, общая база данных и Redis, отдельный режим без межузлового канала — для воспроизведения исходной проблемы.

2 · Диагностика подключения

Узел сообщает свой идентификатор и идентификатор соединения; ключевые события жизненного цикла соединения логируются.

4 · Проверочный клиент

Подключается к разным узлам, фиксирует доставку события и поддерживает четыре сценария проверки.

Программа испытаний

Два режима по одному коду · четыре сценария проверки

Два режима

- Без межузлового канала — воспроизведение проблемы
- С межузловым каналом — проверка решения

Один и тот же код, один и тот же сценарий;
отличие — наличие Redis backplane.

Четыре сценария

- Одиночная доставка между узлами
- Серия из пяти итераций
- Восстановление подписки после разрыва
- Переключение на другой узел после отказа

Главный результат

Один код и один сценарий — разница только в наличии Redis-канала

Без backplane

**ReceiveReportPatch
timed out · 10 000 мс**

**Событие, созданное на app-api-1,
не дошло до клиента на app-api-2**

С backplane

**ok: true · ReceiveReportPatch
получен
deliveryLatencyMs ≈ 9,5 мс**

**Клиент на app-api-2 получил
событие с app-api-1**

Сопоставление двух режимов на одном стенде показывает причинную связь между включением межузлового канала SignalR и успешной доставкой события.

Дополнительные проверки

Повторяемость, восстановление подписки и переключение после отказа узла

Серия из 5 итераций

Успешно: 5 из 5

min 5,5 мс

avg 8,4 мс

p50 9,1 мс

p95 11,6 мс

Подтверждена повторяемость доставки.

Восстановление подписки

После разрыва клиент создаёт новое соединение и снова вступает в группу отчёта.

переподключение 12,9 мс
доставка 6,6 мс

Подтверждена повторная подписка.

Переключение после отказа узла

Один backend остановлен; nginx переключает клиента на другой экземпляр.

переподключение 240,3 мс
доставка 7,3 мс

Подтверждено восстановление после отказа.

Вывод

Ограничение single-node WebSocket-архитектуры устранено за счёт распределённого real-time контура; тезис практики подтверждён экспериментально.

Дополнительный эффект — повышение отказоустойчивости: при отказе одного узла клиенты продолжают получать события через другой экземпляр backend, тогда как single-node архитектура такой возможности не предоставляет.

Проблема

Single-node SignalR: события не пересекают границу узла.

Решение

Несколько экземпляров backend за nginx + Redis backplane; общий код, переключение режима — переменной окружения.

Доказательство

Доставка (без / с) · серия · восстановление подписки · переключение после отказа.